ASSIGNMENT 22

Name: Om Prashant Kulawade

College: Zeal Polytechnic, Narhe, Pune

Branch: Artificial Intelligence and Machine Learning (AIML)

Domain: AIML (Artificial Intelligence and Machine Learning)

https://github.com/omkulawade03/Frontend-Session-Assignments.git

Import Required Libraries

train_model.py > ...

```python
# ========================================
# Import Required Libraries
# ========================================

import pandas as pd          # For loading and handling dataset
import joblib                # For saving and loading ML model

# Import Machine Learning modules
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# Import evaluation metrics
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report
)

```

Q1. (Data Loading & Preprocessing)

Load the Heart Disease dataset.

Separate features into X (independent) and y (dependent/target = HeartDisease).

Perform necessary preprocessing (encoding categorical columns).

Program Code:

```
#Questions 01
```

```python
# ========================================
# Q1. Data Loading & Preprocessing
# ========================================

# Load Heart Disease Dataset
df = pd.read_csv("heart.csv")

# Display first 5 rows of dataset
print("First 5 Records:")
print(df.head())

# Display dataset information
print("\nDataset Information:")
print(df.info())

# Display dataset shape
print("\nDataset Shape:", df.shape)

# ----------------------------------------
# Separate Features (X) and Target (y)
# ----------------------------------------

# Independent variables (Input Features)
X = df.drop("HeartDisease", axis=1)

# Dependent variable (Target)
y = df["HeartDisease"]

# ----------------------------------------
# Encode Categorical Columns
# ----------------------------------------

# Convert categorical columns into numeric format
# drop_first=True avoids dummy variable trap
X = pd.get_dummies(X, drop_first=True)

# Save column names for future prediction
columns = X.columns

print("\nEncoded Feature Columns:")
print(columns)
```

OUTPUT SCREENSHOT:

```
First 5 Records:
   Age Sex ChestPainType  RestingBP  Cholesterol  FastingBS RestingECG  MaxHR ExerciseAngina  Oldpeak ST_Slope  HeartDisease
0   40   M           ATA        140          289          0     Normal    172              N      0.0       Up             0
1   49   F           NAP        160          180          0     Normal    156              N      1.0     Flat             1
2   37   M           ATA        130          283          0         ST     98              N      0.0       Up             0
3   48   F           ASY        138          214          0     Normal    108              Y      1.5     Flat             1
4   54   M           NAP        150          195          0     Normal    122              N      0.0       Up             0
```

```
Dataset Information:
<class 'pandas.DataFrame'>
RangeIndex: 918 entries, 0 to 917
Data columns (total 12 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Age             918 non-null    int64
 1   Sex             918 non-null    str
 2   ChestPainType   918 non-null    str
 3   RestingBP       918 non-null    int64
 4   Cholesterol     918 non-null    int64
 5   FastingBS       918 non-null    int64
 6   RestingECG      918 non-null    str
 7   MaxHR           918 non-null    int64
 8   ExerciseAngina  918 non-null    str
 9   Oldpeak         918 non-null    float64
 10  ST_Slope        918 non-null    str
 11  HeartDisease    918 non-null    int64
dtypes: float64(1), int64(6), str(5)
memory usage: 97.6 KB
None
```

```
Dataset Shape: (918, 12)
```

```
Encoded Feature Columns:
Index(['Age', 'RestingBP', 'Cholesterol', 'FastingBS', 'MaxHR', 'Oldpeak',
       'Sex_M', 'ChestPainType_ATA', 'ChestPainType_NAP', 'ChestPainType_TA',
       'RestingECG_Normal', 'RestingECG_ST', 'ExerciseAngina_Y',
       'ST_Slope_Flat', 'ST_Slope_Up'],
      dtype='str')
```

Q2. (Train-Test Split)

Split the preprocessed data into training and testing sets using train_test_split.

Use test_size=0.20 and random_state=42.

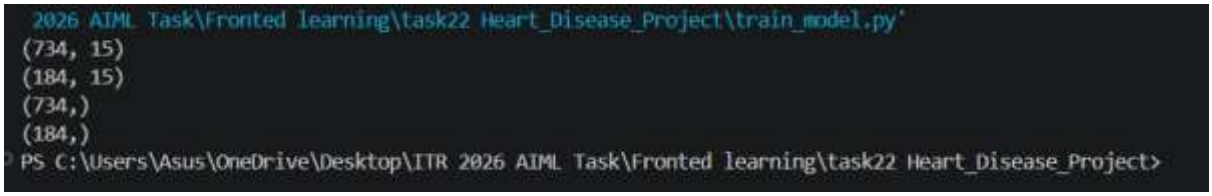
Print the shape of X_train, X_test, y_train, and y_test.

Program Code:

#Questions 02

```
# ========================================
# Q2. Train-Test Split
# ========================================
import pandas as pd
from sklearn.model_selection import train_test_split

# Load dataset
df = pd.read_csv("heart.csv")

# Separate Features and Target
X = df.drop("HeartDisease", axis=1)
y = df["HeartDisease"]

# Encode categorical columns
X = pd.get_dummies(X, drop_first=True)

# Now Split the Data
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)

```

OUTPUT SCREENSHOT:



Q3. (Building Logistic Regression Model)

Train a Logistic Regression model:

Import LogisticRegression from sklearn.

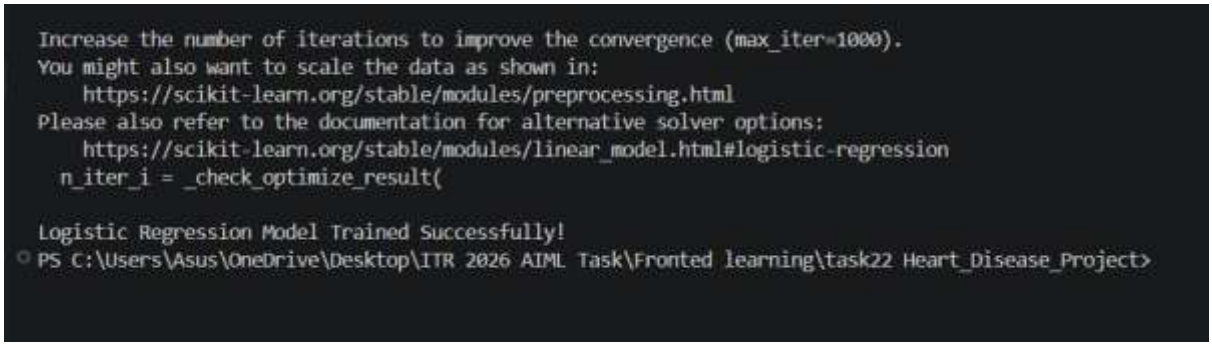
Create the model and fit it on the training data (X_train, y_train).

Program Code:

#Questions 03

```
# ========================================
# Q3. Building Logistic Regression Model
# ========================================

# Import Logistic Regression algorithm
from sklearn.linear_model import LogisticRegression

# ----------------------------------------
# Create the Logistic Regression model
# ----------------------------------------

# max_iter=1000 increases the maximum number
# of iterations to ensure the model converges.
model = LogisticRegression(max_iter=1000)

# ----------------------------------------
# Train (Fit) the Model
# ----------------------------------------

# Train the model using the training dataset
model.fit(X_train, y_train)

# Display success message
print("\nLogistic Regression Model Trained Successfully!")

```

OUTPUT SCREENSHOT:



Q4. (Making Predictions)

Use the trained model to predict on the test data.

Store predictions in y_pred.

Print the first 10 actual values (y_test) and predicted values (y_pred).

Program Code:

#Questions 04

```
# ========================================
# Q4. Making Predictions
# ========================================

# Predict values using testing dataset
y_pred = model.predict(X_test)

# Display first 10 actual values
print("\nFirst 10 Actual Values:")
print(y_test.values[:10])

# Display first 10 predicted values
print("\nFirst 10 Predicted Values:")
print(y_pred[:10])

```

OUTPUT SCREENSHOT:

```
First 10 Actual Values:
[0 1 1 1 0 1 1 0 1 1]

First 10 Predicted Values:
[0 0 1 1 0 1 1 0 1 1]
PS C:\Users\Asus\OneDrive\Desktop\ITR 2026 AIML Task\Fronted learning\task22 Heart_Disease_Project>
```

Q5. (Confusion Matrix)

Create a Confusion Matrix using confusion_matrix(y_test, y_pred).

Display the matrix.

Label TN, FP, FN, and TP in the output.

Program Code:

#Questions 05

```
# ========================================
# Q5. Confusion Matrix
# ========================================

# Generate Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix")
print(cm)

# Extract values from Confusion Matrix
TN = cm[0][0]
FP = cm[0][1]
FN = cm[1][0]
TP = cm[1][1]

# Display Confusion Matrix labels
print("\nTrue Negative (TN):", TN)
print("False Positive (FP):", FP)
print("False Negative (FN):", FN)
print("True Positive (TP):", TP)

```

OUTPUT SCREENSHOT:

Q6. (Model Evaluation Metrics)

Calculate the following metrics:

Accuracy

Precision

Recall

F1 Score

Use classification_report(y_test, y_pred) and print the full report.

Program Code:

#Questions 06

```
# ========================================
# Q6. Model Evaluation
# ========================================

# Calculate Accuracy
accuracy = accuracy_score(y_test, y_pred)

# Calculate Precision
precision = precision_score(y_test, y_pred)

# Calculate Recall
recall = recall_score(y_test, y_pred)

# Calculate F1 Score
f1 = f1_score(y_test, y_pred)

# Display Evaluation Metrics
print("\nModel Evaluation")

print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall   :", recall)
print("F1 Score :", f1)

# Display Classification Report
print("\nClassification Report")
print(classification_report(y_test, y_pred))

```

OUTPUT SCREENSHOT:

```
Model Evaluation
Accuracy : 0.8532608695652174
Precision: 0.9
Recall   : 0.8411214953271028
F1 Score : 0.8695652173913043

Classification Report
              precision    recall  f1-score   support

           0       0.80      0.87      0.83        77
           1       0.90      0.84      0.87       107

    accuracy                           0.85       184
   macro avg       0.85      0.86      0.85       184
weighted avg       0.86      0.85      0.85       184
```

Q7. (Saving the Model)

Save the trained model and preprocessing objects using joblib:

Model → "heart_model.pkl"

Scaler (if used) → "scaler.pkl"

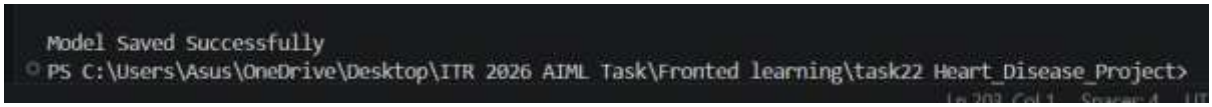
Column list → "columns.pkl"

Program Code:

#Questions 07

```
184  # ========================================
185  # Q7. Save Model
186  # ========================================
187
188  # Save trained Logistic Regression model
189  joblib.dump(model, "heart_model.pkl")
190
191  # Save feature column names
192  joblib.dump(columns, "columns.pkl")
193
194  # Save scaler
195  # (No scaler used in this project, so saving None)
196  joblib.dump(None, "scaler.pkl")
197
198  print("\nModel Saved Successfully")
199
```

OUTPUT SCREENSHOT:



Q8. (Loading & Testing Saved Model)

Load the saved model and objects.

Create sample input data (one patient record).

Preprocess the input (encoding + scaling).

Make a prediction using the loaded model and print the result.

Program Code:

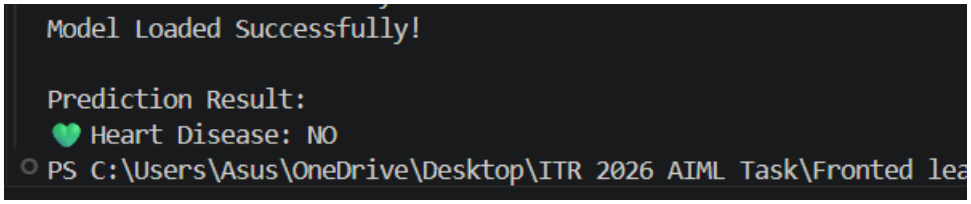
#Questions 08

```
# ========================================
# Q8. Loading & Testing Saved Model
# ========================================

# Import required libraries
import pandas as pd
import joblib

# ----------------------------------------
# Load Saved Model and Objects
# ----------------------------------------

# Load the trained model
loaded_model = joblib.load("heart_model.pkl")

# Load the feature column names
loaded_columns = joblib.load("columns.pkl")

# Load the scaler (None in this project)
loaded_scaler = joblib.load("scaler.pkl")

print("Model Loaded Successfully!")

```

```python
# ----------------------------------------
# Create Sample Input Data
# ----------------------------------------

sample = {
    "Age": 45,
    "Sex": "M",
    "ChestPainType": "ATA",
    "RestingBP": 130,
    "Cholesterol": 230,
    "FastingBS": 0,
    "RestingECG": "Normal",
    "MaxHR": 150,
    "ExerciseAngina": "N",
    "Oldpeak": 1.2,
    "ST_Slope": "Up"
}

# Convert dictionary to DataFrame
sample_df = pd.DataFrame([sample])

```

```python
# ----------------------------------------
# Preprocess the Input Data
# ----------------------------------------

# Apply One-Hot Encoding
sample_df = pd.get_dummies(sample_df)

# Match feature columns with training data
sample_df = sample_df.reindex(columns=loaded_columns, fill_value=0)

# Apply scaling if a scaler exists
if loaded_scaler is not None:
    sample_df = loaded_scaler.transform(sample_df)

# ----------------------------------------
# Make Prediction
# ----------------------------------------

prediction = loaded_model.predict(sample_df)

# Display Prediction Result
print("\nPrediction Result:")

if prediction[0] == 1:
    print("❤️ Heart Disease: YES")
else:
    print("💚 Heart Disease: NO")

```

OUTPUT SCREENSHOT:



Program Code:

#Questions 09

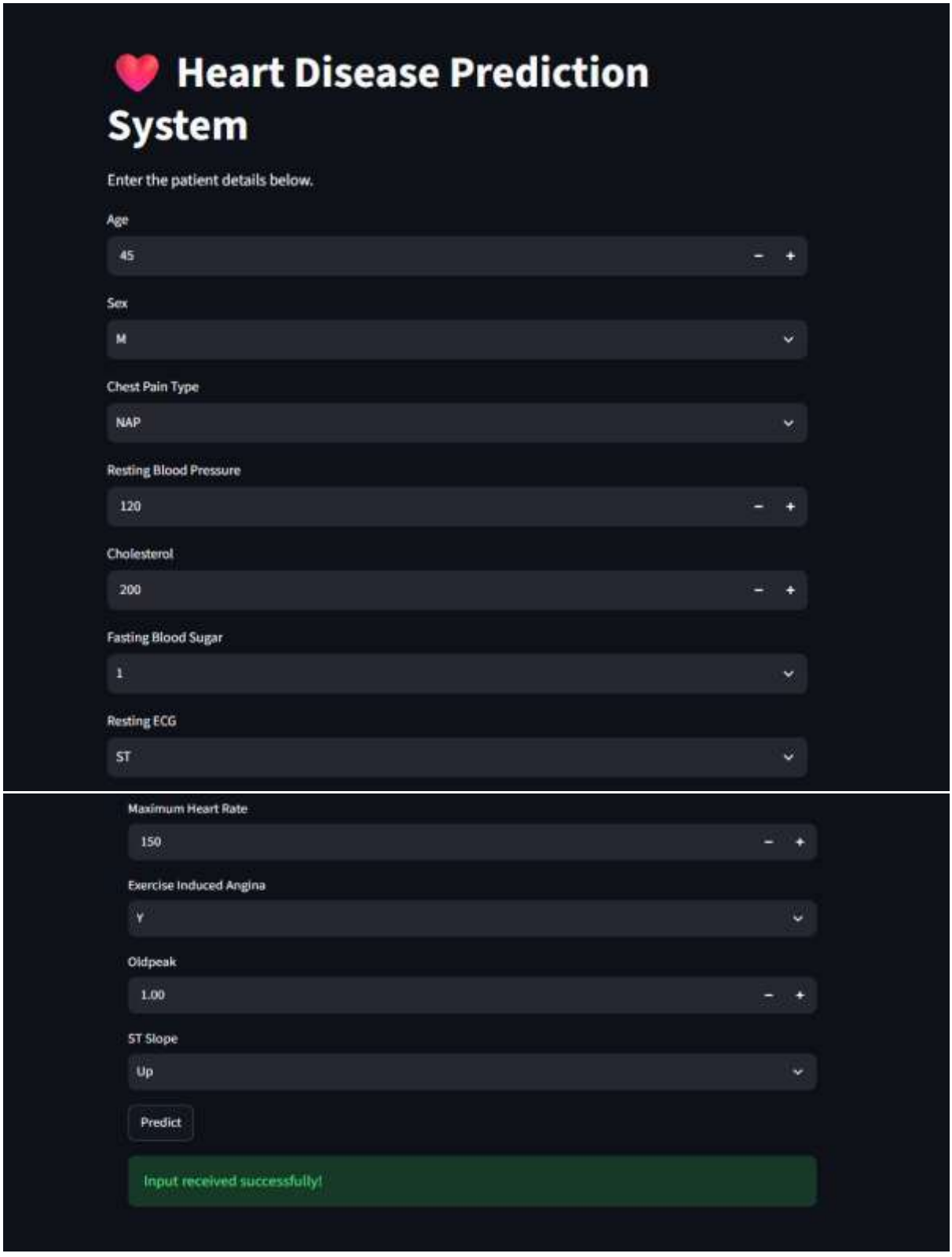
app.py > ...

```python
# ========================================
# Q9. Streamlit Input Interface
# ========================================

import streamlit as st

# Page Configuration
st.set_page_config(page_title="Heart Disease Prediction")

# Title
st.title("❤️ Heart Disease Prediction System")

st.write("Enter the patient details below.")

# ------------------------------
# Input Fields
# ------------------------------

age = st.number_input("Age", min_value=1, max_value=120, value=45)

sex = st.selectbox("Sex", ["M", "F"])

chest_pain = st.selectbox(
    "Chest Pain Type",
    ["ATA", "NAP", "ASY", "TA"]
)

resting_bp = st.number_input(
    "Resting Blood Pressure",
    value=120
)

cholesterol = st.number_input(
    "Cholesterol",
    value=200
)
```

app.py > ...

```python

    fasting_bs = st.selectbox(
        "Fasting Blood Sugar",
        [0, 1]
    )

    resting_ecg = st.selectbox(
        "Resting ECG",
        ["Normal", "ST", "LVH"]
    )

    max_hr = st.number_input(
        "Maximum Heart Rate",
        value=150
    )

    exercise_angina = st.selectbox(
        "Exercise Induced Angina",
        ["Y", "N"]
    )

    oldpeak = st.number_input(
        "Oldpeak",
        value=1.0
    )

    st_slope = st.selectbox(
        "ST Slope",
        ["Up", "Flat", "Down"]
    )

    # ------------------------------
    # Predict Button
    # ------------------------------

    if st.button("Predict"):
        st.success("Input received successfully!")
```

OUTPUT SCREENSHOT:



Q10. (Complete Streamlit Web App)

Complete the Streamlit app for Heart Disease Prediction:

Load the saved model and preprocessing objects.

On button click, preprocess input and show prediction ("Heart Disease: Yes"

or "No") with st.success() or st.error().

Program Code:

```
#Questions 10
```

Q10app.py > ...

```python
# ========================================
# Q10. Complete Streamlit Web App
# ========================================

# Import Required Libraries
import streamlit as st
import pandas as pd
import joblib

# ----------------------------------------
# Load Saved Model and Objects
# ----------------------------------------

model = joblib.load("heart_model.pkl")
columns = joblib.load("columns.pkl")
scaler = joblib.load("scaler.pkl")

# ----------------------------------------
# Streamlit Page
# ----------------------------------------

st.set_page_config(page_title="Heart Disease Prediction")

st.title("💗 Heart Disease Prediction System")
st.write("Enter the patient details below.")

# ----------------------------------------
# Input Fields
# ----------------------------------------

age = st.number_input("Age", min_value=1, max_value=120, value=45)

sex = st.selectbox(
    "Sex",
    ["M", "F"]
)
```

Q10app.py > ...

```python

    chest_pain = st.selectbox(
        "Chest Pain Type",
        ["ATA", "NAP", "ASY", "TA"]
    )

    resting_bp = st.number_input(
        "Resting Blood Pressure",
        value=120
    )

    cholesterol = st.number_input(
        "Cholesterol",
        value=200
    )

    fasting_bs = st.selectbox(
        "Fasting Blood Sugar",
        [0, 1]
    )

    resting_ecg = st.selectbox(
        "Resting ECG",
        ["Normal", "ST", "LVH"]
    )

    max_hr = st.number_input(
        "Maximum Heart Rate",
        value=150
    )

    exercise_angina = st.selectbox(
        "Exercise Induced Angina",
        ["Y", "N"]
    )

```

Q10app.py > ...

```python

    oldpeak = st.number_input(
        "Oldpeak",
        value=1.0
    )

    st_slope = st.selectbox(
        "ST Slope",
        ["Up", "Flat", "Down"]
    )

    # ----------------------------------------
    # Predict Button
    # ----------------------------------------

    if st.button("Predict"):

        # Create patient record
        sample = pd.DataFrame([{
            "Age": age,
            "Sex": sex,
            "ChestPainType": chest_pain,
            "RestingBP": resting_bp,
            "Cholesterol": cholesterol,
            "FastingBS": fasting_bs,
            "RestingECG": resting_ecg,
            "MaxHR": max_hr,
            "ExerciseAngina": exercise_angina,
            "Oldpeak": oldpeak,
            "ST_Slope": st_slope
        }])

        # One-Hot Encoding
        sample = pd.get_dummies(sample)

        # Match columns with training data
        sample = sample.reindex(columns=columns, fill_value=0)
```

```
109
110     # Apply scaler (if used)
111     if scaler is not None:
112         sample = scaler.transform(sample)
113
114     # Predict
115     prediction = model.predict(sample)
116
117     # Display Result
118     if prediction[0] == 1:
119         st.error("💔 Heart Disease: Yes")
120     else:
121         st.success("💚 Heart Disease: No")
122
```

OUTPUT SCREENSHOT: